

AgriTech Sensing Solutions

Continuous Evaluation Project — Review 1

Architecture report and mark-maximising notes, written against the Review 1 rubrics

How the platform works, what evidence in the codebase proves each rubric claim, and the exact fixes that move each component from 7 to 10.

Prepared September 2026 · Spring Boot 3.3.4 / Spring Cloud 2023.0.3 / Java 21

Table of contents

Part I — The project and how it works

- 1 How to use this document
- 2 The system at a glance
- 3 The request lifecycle, end to end
- 4 The automation logic (what makes it more than CRUD)
- 5 Running and demonstrating it

Part II — Rubric-by-rubric notes

- 1 Rubric 1 — Problem Analysis and Requirement Specification
- 2 Rubric 2 — Microservice Identification and Service Discovery
- 3 Rubric 3 — JWT Authentication
- 4 Rubric 4 — API Gateway Configuration
- 5 Rubric 5 — LinkedIn article (out of scope)

Part III — Getting to 10 on every row

- 1 The one-page scoring summary
- 2 Priority list, ordered by marks per hour
- 3 Demo script for the review room
- 4 Viva questions you should expect, with the answers
- 5 Appendix — file inventory for the report

How to use this document

This is two documents stapled together, and both halves are needed for the same purpose.

The first half is the **technical write-up**: what the project is, what every service owns, and how a request actually flows through it. The second half is the **scoring playbook**: for each rubric, the

evidence in your own code that proves the claim, the language that earns the marks, and the honest list of holes an examiner will find if you let them.

Every rubric section uses the same four-part shape:

Part	What it gives you
What the rubric asks	The 0 / 2 / 5 / 7 / 10 ladder, and what the top rung actually means
What your project does	Claims tied to real file paths and real config keys — nothing you cannot defend
How to present it	Sentences and diagrams to put in the report and say in the room
Gap and fix	What is missing for a 10, with code you can paste

Warning — Read this before anything else

Your `README.md` opens by saying the platform is "built to `AGRITECH_PRD.md` and `AGRITECH_UI_UX.md`", and your code comments cite `PRD 7.1`, `PRD 7.2`, `PRD 7.3`, `PRD 8.2`, `PRD 9.3` and `PRD section 16`.

Neither file exists anywhere in the repository. A search across the whole tree returns nothing.

This is the single biggest threat to Rubric 1. A rubric row that says "Deep analysis and Complete Structure" cannot be scored from a README that references a missing specification — the examiner either takes the README's word for it (7 at best) or discovers the gap and reads it as "Basic understanding" (5). Attach the PRD/SRS as an appendix, or write it now from Section 2 of this document. Either way it takes an hour and it is worth five marks.

Rubric 5 (LinkedIn article with DTI concepts) is excluded, as you asked. Section 6 has three lines on it anyway so you do not silently forfeit the row.

Part I — The project and how it works

The system at a glance

AgriTech Sensing Solutions is a precision-agriculture control platform. A fleet of field sensors reports soil moisture, soil temperature, pH and battery level. The platform scores each field's crop health against that crop's optimal envelope, and drives irrigation valves from the score — opening zones when moisture is forecast to fall below the optimal floor, and shutting them off early when moisture reaches the optimal ceiling so it does not waste water.

It is a **Spring Cloud microservices system**: seven Maven modules under one reactor build, four business services behind a reactive API gateway, a Netflix Eureka registry for service discovery, and a browser dashboard that talks only to the gateway.

Module	Port	Owns	Responsibility
eureka-server	8761	—	Service registry; 30 s heartbeat, 90 s eviction
api-gateway	8080	no data	Single entry point: JWT validation, identity headers, routing, rate limiting, CORS
auth-service	8081	User	Login, registration, 24 h JWTs, refresh tokens, revocation list, Google/Supabase federation
sensor-service	8082	SensorDevice, TelemetryReading	Device registry, telemetry ingestion and validation, historical queries
crop-service	8083	Crop, HealthSnapshot	Health scoring, optimal-range evaluation, recommendations, irrigation forecast
irrigation-service	8084	IrrigationSchedule, Valve, IrrigationEvent	Schedules, valve control, moisture cut-out, emergency stop
common	—	no data	Shared error envelope, global exception handler, caller-identity helpers
web	3000	no data	Next.js 16 dashboard (current)
frontend	5173	no data	Vite dashboard (legacy, kept for side-by-side comparison)

Build stack, straight from the root `pom.xml` and module POMs:

Concern	Choice
Language	Java 21
Framework	Spring Boot 3.3.4
Cloud	Spring Cloud 2023.0.3
Gateway	Spring Cloud Gateway (reactive / WebFlux)
Discovery	Netflix Eureka client + server
JWT	jjwt 0.11.5, HMAC-SHA256
Password hashing	BCryptPasswordEncoder (spring-security-crypto)
Persistence	H2 in-memory per service, ddl-auto: update

Concern	Choice
Frontend	Next.js 16 App Router, React 19, Tailwind v4, shadcn, Recharts

Note — One sentence that describes the whole architecture

Every request enters through one gateway that is the only component in the system holding the signing key; the gateway turns a token into trusted headers, and four services that each own their own data collaborate through a registry rather than through each other's hostnames.

The request lifecycle, end to end

This is the diagram to have on screen when the examiner asks "walk me through what happens when a farmer opens the dashboard".

A single authenticated dashboard request, from click to JSON

```

Browser (web :3000)
|
| GET /api/crops   Authorization: Bearer <jwt>
v
api-gateway :8080
|
| [1] RateLimitFilter  order -50   1000/min per user
| [2] JwtAuthFilter    order -100  verify HS256, jti not revoked
| [3] strip client X-User-* -> set verified identity headers
| [4] resolve lb://crop-service .....> Eureka :8761
|      <..... healthy instances
v
crop-service :8083
|
| [5] GET /api/crops + X-User-Email / X-User-Role / X-Farm-Id
| [6] resolve http://sensor-service .....> sensor-service :8082
|      <..... averaged soil moisture / temperature / pH
| [7] score against Crop optimal bands, weights 50/30/20
v
api-gateway :8080
|
v
Browser -> 200 JSON, or one shared error envelope

```

Five things worth saying out loud about this flow, because each one maps to a rubric row:

- The browser never learns a service hostname.** `next.config.ts` rewrites `/api/*` to the gateway, so the origin is single and there is no CORS preflight in the happy path.
- The gateway is the only JWT parser in the system.** Documented verbatim in `api-gateway/src/main/java/com/agritech/gateway/JwtAuthFilter.java:24-28`.
- Discovery is used twice in one request** — gateway to service, then crop-service to sensor-service. Both go through Eureka service-ids, not ports.
- Filters are ordered deliberately:** `JwtAuthFilter` at `-100` runs before `RateLimitFilter` at `-50`, so the rate limiter can distinguish an authenticated user quota from an anonymous IP quota.

5 **Failures look the same everywhere.** One envelope, produced by `common/ApiExceptionHandler.java`, per the PRD's error contract.

The automation logic (what makes it more than CRUD)

This is the part that separates a "microservices demo" from a "system", so it belongs in the report even though no rubric row names it directly.

Health score. For each of soil moisture, soil temperature and pH: 100 inside the crop's optimal band, losing points in proportion to how far outside it the reading sits, measured in *band widths* rather than raw units. The three are weighted 50 / 30 / 20 toward moisture. A drift of up to 35 % of a band width is a warning; beyond that is critical. Implemented in `crop-service/.../HealthScore.java`, covered by `HealthScoreTest`.

Irrigation forecast. The moisture slope over the last six hours is extrapolated forward to the moment it would cross below the optimal floor. Only forecasts inside a 48-hour horizon are reported — beyond that, linear extrapolation of a noisy signal is not worth showing the farmer.

Water saving. Every schedule defaults to `skipIfMoist`. Before opening, irrigation-service asks crop-service for the field's health and *skips the run* if moisture is already at or above the optimal ceiling. A zone already running shuts itself off mid-run when it reaches that ceiling. This is the closed loop, and it is the strongest answer to "why does this need to be more than one service?".

Demo closure. `sensor-service/DeviceSimulator.java` seeds 13 devices across 4 fields, backfills 48 hours of 15-minute samples on a sine daily cycle, then emits every 30 seconds. One device is deliberately left offline and one field starts dry, so the alert paths are visible without waiting for a failure. When a valve opens, irrigation-service calls `POST /api/telemetry/field/{id}/irrigation-event`, and simulated moisture climbs ~1.4 per tick while watering — so the score visibly recovers on screen. Set `SIMULATOR_ENABLED=false` to run against real devices; they post to the same `POST /api/telemetry/ingest` contract.

Running and demonstrating it

```
./scripts/stack.ps1 start      # build all modules, then launch in dep order
npm install --prefix web
npm run dev --prefix web      # http://localhost:3000
```

`stack.ps1` loads the git-ignored root `.env` into the child process environment, then starts `eureka(8761) → auth(8081) → sensor(8082) → crop(8083) → irrigation(8084) → gateway(8080)` as `java -jar <module>/target/<name>-1.0.0.jar`, blocking on each service's `/actuator/health` returning

UP (90 s timeout) and writing logs to `.run/`. `-SkipBuild` skips `mvnw install -DskipTests` — which

means **backend edits silently run as stale JARs unless you reinstall first.**

Credential	Role	Sees
admin@agritech.io / admin1234	ADMIN	Full sensor, schedule and valve control
farmer@agritech.io / farmer1234	FARMER	Read-only telemetry and crop health

The dashboard never asks which role you are; navigation and controls are filtered from the `role` claim inside the issued token. That is a good line to say in a viva — it means authorisation is driven by the token, not by a UI setting.

Verification, both levels:

```
./mvnw test           # 3 unit test classes
bash scripts/smoke-test.sh # end-to-end against a running stack
```

The smoke test covers the PRD's section 9.3 scenarios: unauthenticated rejection, login for both roles, forged-token rejection, the farmer/admin authorisation split, device registration, telemetry ingestion including out-of-range rejection, health scoring from live telemetry, cross-service trend retrieval through Eureka, manual valve control, emergency stop, schedule CRUD, and logout revocation taking effect

at the gateway. **Run this live in the review.** It is the only artifact that proves all four rubric rows at once.

Give the services ~30 seconds after startup before the first cross-service call — that is Eureka's registry-fetch interval, and until it completes the gateway has no route targets. Know this before the demo, because a 502 in the first few seconds is expected behaviour and not a bug.

Part II — Rubric-by-rubric notes

Rubric 1 — Problem Analysis and Requirement Specification

The ladder. 0 no understanding · 2 limited understanding · 5 basic understanding · 7 good analysis and well-defined requirements · 10 deep analysis and complete structure.

What "complete structure" is actually testing

Rows scored on *structure* are marked off a checklist. For a requirement specification the examiner's mental checklist is almost always:

- 1 A problem statement with a **quantified cost**, not a topic
- 2 Named stakeholders with **different** needs
- 3 Numbered, **testable** functional requirements
- 4 Non-functional requirements expressed as **measurable targets**
- 5 Explicit scope boundary — what is deliberately *not* being built
- 6 Constraints and assumptions

7 A **traceability matrix** linking each requirement to the artifact that satisfies it

Most student submissions do 1–4 and stop. Items 5, 6 and 7 are what separates 7 from 10 — and your project is unusually well-positioned for them, because you already have the raw material.

What your project already does

You have real, citable evidence for items 1–6:

Checklist item	Evidence already in the repo
Quantified problem	README: telemetry in, health scored against optimal envelope, valves driven from the score. Water saving is a stated design goal.
Distinct stakeholders	Two roles with genuinely different permissions: ADMIN mutates devices/schedules/valves, FARMER reads. Enforced in <code>CallerContext.requireAdmin</code> .
Numbered FRs	Code comments literally cite FR-1 (<code>JwtService.issue</code>) and FR-2 (<code>CallerContext.requireAdmin</code>)
Sectioned spec	Comments cite PRD 7.1 routing, PRD 7.2 rate limits, PRD 7.3 error envelope, PRD 8.2 load balancing, PRD 9.3 test scenarios, PRD 16 Phase 2+
Explicit non-goals	README "Deliberate simplifications" table lists seven cut corners <i>with their ceiling and their upgrade path</i>
Measurable NFRs	Concrete numbers already exist: 24 h token TTL, 30-day refresh TTL, 100/1000/100 requests-per-minute tiers, 30 s heartbeat, 90 s eviction, 15 min offline threshold, 20 % battery threshold, 48 h forecast horizon, WCAG AA 4.5:1 contrast

Do this — The cheapest 7-to-10 move in this entire document

Your README's "Deliberate simplifications" table — *simplification* → *ceiling* → *upgrade* — is exactly the kind of artifact that reads as "deep analysis". It shows you know where the design breaks. Promote it from the README into the requirements section of your report, rename it "**Scope Boundaries and Architectural Debt**", and cite the code comments that mark each one. Very few submissions in a cohort can do this honestly; almost none do it with file-level references.

The traceability matrix to put in your report

This is the item-7 artifact. It is already 90 % derivable from your code — the numbering below is reconstructed from the endpoints and the `FR-*` / `PRD x.x` comments found in source. Present it as a table; it is the single strongest page in the submission.

Req	Requirement	Where satisfied	Verifiable by
FR-1	Issue a 24 h access token carrying role and farm identity	<code>auth-service/.../JwtService.issue()</code>	decode token, inspect claims
FR-2	Restrict device, schedule and valve mutation to ADMIN	<code>common/.../CallerContext.requireAdmin()</code>	smoke test: farmer POST <code>/api/sensors/register</code> → 403
FR-3	Register and de-commission field sensors	<code>sensor-service</code> POST/PUT/DELETE <code>/api/sensors/{deviceId}</code>	API call
FR-4	Ingest bulk telemetry, validating each reading independently	<code>TelemetryController</code> POST <code>/api/telemetry/ingest</code>	out-of-range sample rejected without failing the batch
FR-5	Retrieve history over selectable windows (1h...90d)	GET <code>/api/telemetry/{deviceId}?range=</code>	API call
FR-6	Score crop health against per-crop optimal bands	<code>crop-service/.../HealthScore.java</code>	<code>HealthScoreTest</code>
FR-7	Forecast time-to-irrigation within a 48 h horizon	<code>HealthSnapshot.hoursToIrrigation</code>	dashboard trend panel
FR-8	Schedule irrigation as DAILY or WEEKLY with on/off toggle	<code>irrigation-service</code> <code>IrrigationSchedule</code> , PATCH <code>.../active</code>	API + UI toggle
FR-9	Manual valve control bounded to 1–240 minutes	<code>ValveController</code> POST <code>/api/valves/{id}/open?durationMinutes=</code>	boundary test at 0 and 241
FR-10	Global emergency stop closes every open valve	POST <code>/api/valves/emergency-stop</code>	smoke test
FR-11	Skip or cut off watering when moisture is already optimal	<code>skipIfMoist</code> + mid-run ceiling check	closed-loop demo
FR-12	Revoke a session at the edge on logout	POST <code>/api/auth/logout</code> → <code>RevocationCache</code>	logout, replay old token → 401 within 15 s
FR-13	Federated Google sign-in for first-time users	POST <code>/api/auth/google</code> + <code>SupabaseVerifier</code>	Google login provisions FARMER on FARM-001
NFR-1	Single entry point; no direct service exposure	<code>api-gateway</code> route table, PRD 7.1	architecture diagram
NFR-2	Route capacity tiers	PRD 7.2: 100 public / 1000 authenticated / 100 device per minute	<code>RateLimitFilter</code>
NFR-3	Uniform error contract	PRD 7.3: <code>common/ApiExceptionHandler</code>	any 4xx response

Req	Requirement	Where satisfied	Verifiable by
NFR-4	Instances added remove downtime	PRD 8.2: <code>lb://</code> round-robin over healthy instances	run two sensor instances, watch Eureka
NFR-5	Detect a dead device within 15 minutes	<code>SensorDevice.health()</code>	seeded dark device shows OFFLINE
NFR-6	WCAG AA contrast in both themes	README design-system notes, worst case 4.63:1	contrast audit

Gap and fix

Gap	Why it costs marks	Fix
AGRITECH_PRD.md and AGRITECH_UI_UX.md are referenced but absent	The whole specification claim rests on an unverifiable citation	Attach the PRD as Appendix A, or reconstruct it from the matrix above. If the PRD truly never existed, delete the citations — an examiner who checks one reference and finds it missing stops trusting all of them.
No problem statement with numbers	"Precision agriculture platform" is a topic, not a problem	Open with a quantified sentence: irrigation is roughly 70 % of agricultural freshwater use, and schedule-based watering ignores current soil state, so water and yield are both lost. State the target: water applied per hectare, and the count of fields a single operator can supervise.
No measurable NFR targets for latency/throughput/availability	"Scalable and secure" is untestable	Add a table with numbers you can defend: p95 gateway overhead under 20 ms, telemetry ingestion sustaining 1 reading/s/device, 99 % API availability, stateless horizontal scale of any service behind Eureka.
No use-case or user-story set	Requirement <i>structure</i> includes actors → scenarios	Three actors (Farm Operator, Field Technician/ADMIN, Device), five use cases each, one ASCII or Mermaid use-case diagram.
Roles limited to two, no tenancy model	A farm platform with one <code>farmId</code> per user invites the question "what about multi-tenant farms?"	Either add the tenancy requirement explicitly to scope, or state it as a Phase 2 non-goal — do not leave it unaddressed.
Only 3 unit test classes, none for gateway/sensor/common	"Well-defined requirements" implies each is verifiable	Add tests for <code>JwtAuthFilter</code> (accept, reject forged, reject revoked, strip headers) and <code>TelemetryController</code> validation bounds. These are the two highest-value test files for this rubric.

Rubric 2 — Microservice Identification and Service Discovery

The ladder. 0 not identified · 2 few services defined · 5 basic service split · 7 well-structured services · 10 highly modular design.

What "highly modular" is actually testing

Three questions decide this row: *why do these boundaries exist, does each service own its data, and can one service change or scale without touching the others*. If your decomposition story is "the tutorial had these folders", you are at 5. If it is "each boundary is a business capability with its own data and its own scaling profile", you are at 10.

What your project does — the decomposition story

You have **four business services plus three infrastructure/shared modules**, and the boundaries are defensible because each one is drawn on a different axis:

Service	Business capability	Owns	Scaling driver	Change cadence
auth-service	Identity and access	User	login bursts, negligible per-request cost	low — security code should be boring
sensor-service	Device telemetry	SensorDevice, TelemetryReading	write volume — scales with device count	low
crop-service	Agronomic analysis	Crop, HealthSnapshot	CPU — scoring and forecasting	high — the model changes often
irrigation-service	Actuation	IrrigationSchedule, Valve, IrrigationEvent	low volume, but safety-critical	medium

Service dependencies — synchronous, through Eureka service-ids

```
Dashboard    web :3000    (legacy frontend :5173)
  |
  v
+-----+
|      api-gateway :8080      |
|  JWT verify, identity headers, rate  |
|  limit, CORS, lb:// routing  |
+-----+
  v           v           v           v
auth-service  sensor-  crop-  irrigation-
:8081         service  service  service
:8082         :8082    :8083    :8084
```

All five register with Eureka :8761 (30s heartbeat, 90s eviction)
common (shared JAR) = error envelope + CallerContext identity

Every synchronous service-to-service call, with its purpose:

Caller	Callee	Call	Why
Dashboard	api-gateway	GET /api/crops	All UI traffic; one origin, no preflight
api-gateway	auth-service	GET /api/auth/revoked, polled every 15 s	Revoked token ids
crop-service	sensor-service	GET /api/telemetry/field/{id}/latest	Averaged moisture, temperature, pH
irrigation-service	crop-service	GET /api/crops/{id}/health	Decide whether to skip a run
irrigation-service	sensor-service	POST ../field/{id}/irrigation-event	Signal that a zone is watering

The dependency graph is **acyclic**, and there are no hardcoded localhost ports in any route or client.

Note the one edge that is *not* a business call: `auth-service` → `gateway` for the revocation list. Be ready to defend it. The honest answer is that it is a pragmatic trade — the gateway needs revoked-token ids, and rather than share a database it polls a read-only endpoint. The upgrade is a shared Redis revocation set, and your README already says so.

Say this in the room: *the split is not by screen, it is by scaling profile and by blast radius.*

Sensor-service needs to scale with 10,000 devices writing; crop-service needs CPU for analysis; irrigation-service needs neither, but it is the only service that can physically waste water or stress a crop, so it is isolated so that a bad crop-service deploy cannot reach a valve. That is a modular design argument, not a folder-layout argument.

Data ownership is real. Each service has its own H2 datasource and its own entities. There is no shared database and no cross-service foreign key. When crop-service needs soil moisture it calls `GET /api/telemetry/field/{id}/latest` — it does not read the sensor table. This is the single most important modularity claim you have; most submissions fail it by sharing one schema.

Service discovery — the actual mechanics

This is where you can clearly out-score the cohort, because you can show the config rather than describe the concept.

Server. `@EnableEurekaServer` on port 8761 (`eureka-server`).

Clients. Every service carries `spring-cloud-starter-netflix-eureka-client` and registers via config — no `@EnabledDiscoveryClient` annotation is needed on the Spring Cloud 2023 line, and the dependency plus these keys do the work:

```
eureka:
  client:
    service-url:
      defaultZone: ${EUREKA_URL:http://localhost:8761/eureka}
  instance:
    prefer-ip-address: true
```

`prefer-ip-address: true` matters: it makes the registry hand back an IP rather than a reverse-DNS hostname, which is what lets the stack run on a laptop without a DNS server.

Consumers use logical names, never hostnames. Gateway routes resolve through the registry:

```
- id: sensor-service
  uri: lb://sensor-service
  predicates:
    - Path=/api/sensors/**,/api/telemetry/**
```

Inter-service clients do the same with a `@LoadBalanced` template — `SensorClient.java:25` calls `http://sensor-service/...`, `FarmClient.java:29,41` call `http://crop-service/...` and

`http://sensor-service/...`. **There is not a single hardcoded `localhost:8080` in any route or client in**

the codebase. Say that sentence out loud; a lot of submissions claim service discovery and then route to fixed ports.

Load balancing. `lb://` URIs are round-robin across healthy instances, which is the PRD 8.2 requirement. This is demonstrable: start a second `sensor-service` on another port, watch both appear under the Eureka UI at `:8761`, and show the gateway alternating between them.

Registry timings to quote: 30 s heartbeat, 90 s eviction, 30 s client registry fetch. The last one is the reason a fresh stack returns 502 for the first ~30 seconds — quote it as a *known trade-off of AP eventual consistency in Eureka* rather than as a bug. That reframing alone signals real understanding.

Note — The theory sentence that lifts this row

Eureka is an AP system in CAP terms: it prefers availability and partition tolerance over a strongly consistent registry view. That is the correct choice for service discovery, because routing to an instance that just died is recoverable with a retry, whereas refusing to route at all is not. The 30 s registry-fetch interval is the visible price of that choice.

Gap and fix

Gap	Why it costs marks	Fix
No resilience between services — only try/catch plus 2 s/4 s timeouts, and the code itself admits it via ponytail: comments at <code>SensorClient.java:17-19</code>	"Highly modular" implies failures are contained. Right now crop-service degrades only as gracefully as a catch block.	Add <code>spring-cloud-starter-circuitbreaker-resilience4j</code> : circuit breaker on <code>SensorClient/FarmClient</code> , a fallback URI on each gateway route, and a cached last-known-good health snapshot in crop-service. Then <i>demonstrate</i> it by killing sensor-service mid-demo.

Gap	Why it costs marks	Fix
H2 in-memory per service, <code>ddl-auto: update</code>	A distributed-database-per-service design that loses all data on restart reads as unfinished, and it undermines the "each service owns its data" claim	Point <code>spring.datasource.*</code> at PostgreSQL with one schema per service (not one shared schema) so the boundary is enforced by the DB. Add Flyway migrations — this also fixes "no schema versioning".
No centralised configuration; secrets duplicated per service YAML	Modularity claim weakens when six files must change to rotate a key	Spring Cloud Config Server, or at minimum a single <code>AGRITECH_JWT_SECRET</code> env var sourced in one place. Note: the dev HMAC default is currently duplicated in both gateway and auth YAML.
No eventing — health analysis is a 5-minute poll	Examiners ask "why is this not event-driven?"	You already list Kafka in the README's simplification table. Promote it to a named Phase 2 requirement (NFR/FR) with a rationale: telemetry ingestion should publish to a topic and crop-service should consume, decoupling write spikes from analysis.
No distributed tracing across four hops	You cannot <i>show</i> the call graph, only assert it	Add Micrometer Tracing + Zipkin. A single trace spanning gateway → crop → sensor is the most persuasive possible screenshot for this rubric row.
Eureka is a single node	"What if the registry dies?" is a certain question	Answer verbally: Eureka clients cache the last registry locally, so services keep talking to each other if the server dies; the registry is not on the request path. Then note peer replication as the production upgrade. Knowing this is worth real marks.
No Docker / compose / Kubernetes	Discovery is demonstrated on one laptop with fixed ports	A <code>docker-compose.yml</code> with 7 services plus Eureka proves location transparency far better than any paragraph. High marks-per-hour.

Rubric 3 — JWT Authentication

The ladder. 0 not implemented · 2 partial setup · 5 basic authentication · 7 secure implementation · 10 robust and secure.

What "robust and secure" is actually testing

7 means the token works and is verified. 10 means the *lifecycle* is handled — issuance, expiry, refresh, revocation, role propagation, key management, and the attack surface — and that you can name the threats you defended against. Expect to be pushed on: replay, forged roles, token leakage, algorithm confusion, and what happens between logout and the next poll.

What your project does

Issuance. `auth-service` verifies credentials with `UserRepository.findByEmail` plus `BCryptPasswordEncoder.matches`, then mints a signed token:

```
// auth-service/.../JwtService.issue() - FR-1
Jwts.builder()
    .setId(UUID.randomUUID().toString()) // jti - the revocation handle
    .setSubject(user.getEmail())         // sub
    .claim("role", user.getRole())
    .claim("farmId", user.getFarmId())
    .claim("name", user.getDisplayName())
    .setIssuedAt(now)
    .setExpiration(new Date(now.getTime() + accessTtlMs))
    .signWith(key, SignatureAlgorithm.HS256)
    .compact();
```

Claim	Value	Why it is there
jti	random UUID	Lets a stateless token be individually revoked
sub	email	Stable user identity
role	ADMIN / FARMER	Drives authorisation without a DB round-trip
farmId	e.g. FARM-001	Tenant scoping
name	display name	UI only
iat / exp	issued / expiry	24 h TTL, configurable via <code>access-ttl-hours</code>

Validation happens in exactly one place. `JwtAuthFilter` is a `GlobalFilter` at order `-100`. It short-circuits `OPTIONS` and a small explicit public-path list, parses the bearer token, checks the revocation cache, and then converts identity into headers:

```
private static final List<String> PUBLIC_PATHS = List.of(
    "/api/auth/login",
    "/api/auth/register",
    "/api/auth/refresh",
    "/api/auth/google",
    "/actuator/health");

ServerHttpRequest forwarded = request.mutate()
    .headers(h -> IDENTITY_HEADERS.forEach(h::remove)) // anti-forgery
    .header("X-User-Email", claims.getSubject())
    .header("X-User-Role", String.valueOf(claims.get("role")))
    .header("X-Farm-Id", String.valueOf(claims.get("farmId")))
    .build();
```

Do this — The detail that proves you understand the design

The order of those two operations is the whole security model. Client-supplied `X-User-*` headers are removed **before** the verified values are written. Without that line, any caller could send `X-User-Role: ADMIN` and be trusted by every downstream service. Lead with this when asked how roles are propagated — it is the difference between "we used JWTs" and "we understood the trust boundary".

Authorisation. Downstream, `common/CallerContext.requireAdmin()` throws a 403 `ApiException` unless

`X-User-Role` is `ADMIN`. Missing role defaults to `FARMER` — fail-closed to the *lower* privilege, which is the correct default and worth stating explicitly.

Refresh and revocation. Access token 24 h; refresh token is an opaque UUID held in an in-memory `TokenStore` with a 30-day TTL, exchanged at `POST /api/auth/refresh`. `POST /api/auth/logout` revokes both the access token's `jti` and the refresh token. The gateway learns about revocations by polling `lb://auth-service /api/auth/revoked` every 15 s (`RevocationCache`, `agritech.jwt.revocation-poll-ms`).

Federated Google login. A genuinely non-textbook addition, and it is the strongest "robust" evidence you have. The frontend performs OAuth through `supabase-js`; the resulting Supabase token is posted to `POST /api/auth/google`; `SupabaseVerifier` **introspects** it by calling `{SUPABASE_URL}/auth/v1/user` rather than verifying it locally; first-time users are auto-provisioned as `FARMER` on `FARM-001`; and the platform then issues its *own* internal JWT, so the rest of the system is completely unaware that Google was involved.

That last point is the answer to "why not just use the Supabase token everywhere?". Internal tokens carry `role` and `farmId`, keep the gateway's verification path single and fast, and let you swap identity providers without touching four services. This is standard identity-brokering and it is worth naming as such.

Warning — There is a real technical reason behind the introspection — use it

An earlier version of `SupabaseVerifier` verified Supabase tokens locally with the legacy HS256 JWT secret (the `.bak` files at the repo root still hold it). Supabase moved to ES256/JWKS-signed tokens, so local HS256 verification began returning 401. Introspecting `GET /auth/v1/user` works regardless of the signing algorithm. Present this as a *discovered constraint and a deliberate design response* — it is one of the few genuinely production-shaped lessons in the project, and it makes the "robust" claim concrete rather than borrowed.

Tested. `auth-service/src/test/java/.../SupabaseVerifierTest.java` runs 7 tests against an embedded `HttpServer` — real tests, no network, no mocks of the thing under test.

Threat model — the table that gets you the 10

Examiners rarely ask "is it secure". They probe one specific attack. Put this table in the report and you have pre-answered all of them.

Threat	Status	Evidence / honest gap
Password disclosure	Defended	BCrypt hashes; <code>passwordHash</code> is nullable so federated users legitimately have none
Forged role via header injection	Defended	<code>IDENTITY_HEADERS.forEach(h : : remove)</code> before setting verified values
Unsigned / tampered token replay	Defended	<code>parseClaimsJws</code> throws on signature mismatch; <code>JwtException</code> → 401 <code>invalid_token</code> ; covered by smoke test
Stolen-but-valid token after logout	Partly defended	<code>jti</code> revocation list, but a window of up to 15 s exists by design. Name the window; do not hide it.
<code>alg: none</code> / algorithm-confusion downgrade	Defended by construction	The parser is given a <code>SecretKey</code> and pins HS256 at signing time; it will not negotiate an algorithm
Long-lived access token leakage	Gap	24 h access TTL is long. Shorten to 15 min and rely on the existing refresh flow — this is a one-line config change with outsized marks.
Weak / shared signing secret	Gap	<code>agritech-dev-secret-key-change-me-in-production-0123456789</code> is a default in <i>both</i> gateway and auth YAML. Fine for a demo, but you must say so and name the fix (env-var injection plus asymmetric keys plus rotation).
Header trust if a service is reachable directly	Gap — and the most likely question	Downstream services trust <code>X-User-Role</code> because they are assumed unreachable except via the gateway. Nothing in the repo enforces that. Fix: network policy, mTLS, or an internal HMAC on forwarded headers.

Threat	Status	Evidence / honest gap
No Spring Security filter chain	Gap	Only <code>spring-security-crypto</code> is on the classpath. Authorisation is header-string comparison, not <code>@PreAuthorize</code> .
CORS wildcard with credentials	Gap	<code>allowedOriginPatterns: http://localhost:*</code> with <code>allowCredentials: true</code> is convenient for development and would be a finding in production. State it as a dev-only setting.
Secrets in the working tree	Checked — clean	Root <code>.env</code> and <code>frontend/.env</code> hold a live Supabase project URL and anon key, but <code>.gitignore</code> covers <code>.env</code> , <code>**/.env</code> and <code>.env.*.local</code> , and <code>git ls-files</code> confirms neither is tracked. <code>.env.example</code> is the committed template.

Gap and fix, ranked by marks per hour

Priority	Fix	Cost
1	Drop access TTL from 24 h to 15 min; the refresh endpoint and <code>TokenStore</code> already exist	1 config line
2	Add route-level or method-level role enforcement in services — <code>@PreAuthorize("hasRole('ADMIN')")</code> , or at minimum a <code>RequireAdmin</code> interceptor — so authorisation does not depend on a header string	~1 hour
3	Remove the duplicated HMAC default; require <code>AGRITECH_JWT_SECRET</code> to be present and fail startup if absent	15 min
4	Move <code>TokenStore</code> and the revocation set to Redis, closing both the 15 s window and the restart-logs-everyone-out limitation	2–3 hours
5	Add refresh-token rotation and reuse detection (a presented-then-used refresh token invalidates its family)	2 hours
6	Migrate HS256 → RS256/ES256 so the gateway can hold only a public key and cannot mint tokens	2 hours — and it directly answers the "shared secret" objection

Priority	Fix	Cost
7	Enforce that services are reachable only via the gateway (Docker network / mTLS / internal header signature)	with the Docker work

Rubric 4 — API Gateway Configuration

The ladder. 0 not configured · 2 partial routing · 5 basic routing · 7 proper configuration · 10 optimised and secure.

What "optimised and secure" is actually testing

7 is a working route table. 10 adds the cross-cutting concerns that a gateway exists to own: rate limiting, resilience, observability, request/response shaping, security headers, and a documented reason

for every line of config. The word *optimised* invites you to talk about the reactive stack, connection reuse, caching, and load distribution.

What your project does

Route table. Four routes, all discovery-resolved, in `api-gateway/src/main/resources/application.yml`:

id	uri	predicates
auth-service	lb://auth-service	Path=/api/auth/**
sensor-service	lb://sensor-service	Path=/api/sensors/**,/api/telemetry/**
crop-service	lb://crop-service	Path=/api/crops/**
irrigation-service	lb://irrigation-service	Path=/api/irrigation/**,/api/valves/**

Two deliberate choices worth explaining:

- `spring.cloud.gateway.discovery.locator.enabled: false`. Auto-discovery would publish every registered service under a `/servicename/**` path, exposing internal services to the internet through the gateway. Explicit routes act as an allow-list. This is a security decision, and saying so converts a config line into a rubric mark.
- **No StripPrefix**. The gateway forwards the path unchanged, and downstream controllers map the full `/api/...` path. This keeps the external and internal contract identical, so a service log line matches what the browser sent. Many projects add `StripPrefix=1` and then have divergent paths in logs; your choice is defensible either way — the point is that you can justify it.

Filter chain, in execution order.

Order	Component	Responsibility
-100	JwtAuthFilter	Verify HS256, check revocation, strip and set identity headers
-50	RateLimitFilter	Three-tier fixed-window limiting, emits X-RateLimit - *
background	RevocationCache	Polls lb://auth-service /api/auth/revoked every 15 s
shared	GatewayResponses	One error envelope for gateway- and service-originated failures

Auth running *before* rate limiting is what allows the limiter to key on the authenticated user rather than only on IP. That ordering is a design decision, not an accident.

Rate limiting, per PRD 7.2.

Caller	Key	Limit
Anonymous	client IP	100 / min
Authenticated	user	1000 / min
Device	X-Device-Id	100 / min

CORS is terminated at the gateway for `http://localhost:*` and `http://127.0.0.1:*` across all methods, exposing `X-RateLimit-Limit` and `X-RateLimit-Remaining`. In the normal path the browser never

sees a preflight, because `next.config.ts` rewrites `/api/*` to the gateway and keeps a single origin

—

gateway CORS is the fallback for the legacy Vite app on 5173.

Observability. `management.endpoints.web.exposure.include: health,info,gateway` — the `/actuator/gateway/routes`

endpoint dumps the live route table, and `logging.level.AUDIT: INFO` gives a named audit logger that

records rejections (`audit.warn("auth REJECTED path={} reason={}", ...)` in `JwtAuthFilter:72`).

Health-gated startup. `stack.ps1` blocks on each service's `/actuator/health` returning `UP` before launching the next, so the gateway never starts against an empty registry.

Gap and fix

Gap	Why it costs marks	Fix
No circuit breaker or fallback URI on any route	"Optimised" implies graceful degradation. Today a dead crop-service returns a raw 503.	Add <code>spring-cloud-starter-circuitbreaker-resilience4j</code> and a <code>fallbackUri: forward:/fallback/crop</code> per route returning the cached last-known-good snapshot. Then demo it by killing a service.
In-memory fixed-window counters	The README already admits the ceiling: N gateway replicas allow N× the quota	Redis-backed <code>RequestRateLimiter</code> with the <code>principal-name</code> key resolver. Also gives you sliding-window accuracy.
No response caching	Telemetry rollups and crop lists change on a 15-minute cadence but are fetched constantly	Add a short-TTL cache for <code>GET /api/crops</code> and <code>/api/telemetry/field/*/latest</code> , or an <code>s-maxage</code> header. This is the most literal reading of "optimised" and it is cheap.
No distributed tracing / correlation id	You cannot show a request crossing four services	Micrometer Tracing + Zipkin; propagate a <code>X-Request-Id</code> in a global filter regardless.
No security response headers	A gateway is the natural place for them	<code>DefaultSecurityHeaders</code> filter: <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , <code>Strict-Transport-Security</code> , <code>Referrer-Policy</code> , <code>CSP</code> .
No HTTPS / TLS	Everything is plain HTTP	Add a keystore and <code>server.ssl.*</code> on the gateway, or terminate TLS in front. Even a self-signed cert lets you <i>demonstrate</i> an encrypted edge.
No API documentation at the gateway	The examiner cannot explore the API without your README table	<code>springdoc-openapi</code> aggregated across services, served at <code>/swagger-ui.html</code> . High marks-per-hour; roughly 30 minutes.
No request/response logging or payload-size limits	Robustness detail	<code>spring.codec.max-in-memory-size</code> plus a logging filter.
Public-path matching uses <code>startsWith</code>	<code>/api/auth/loginish</code> also matches, and <code>/actuator/health</code> is exposed at the edge. Low real risk, but it is the kind of thing a sharp examiner spots	Use exact matches or <code>Path</code> predicates, and restrict actuator exposure to <code>/actuator/health</code> only.

Rubric 5 — LinkedIn article (out of scope, three lines)

You asked me to skip this. Since it is still a scored row, three sentences: the DTI angle your project actually supports is *closed-loop automation on live sensor data* — the system decides, actuates, and re-measures, which is the definition of precision agriculture rather than dashboards. Structure it as problem → architecture → one concrete water-saving number → what you would do next. Publish it before the

review, not during; "Modules and System Architecture only Present" is a 2, and the article is 10 marks for an hour of writing.

Part III — Getting to 10 on every row

The one-page scoring summary

Rubric	Realistic today	After the top 3 fixes	What is holding the top rung back
1 — Problem analysis and requirements	7	10	The referenced PRD does not exist in the repo; no traceability matrix; NFRs are not stated as targets
2 — Microservices and discovery	8	10	Genuinely good discovery and boundaries; held back by in-memory H2, no resilience, no containers
3 — JWT authentication	7	10	Solid lifecycle and a real federation story; held back by header-trust, 24 h TTL, shared dev secret
4 — API gateway	7	10	Correct routes, ordered filters, three-tier limiting; held back by no circuit breaker, no caching, no tracing, no docs

Priority list, ordered by marks per hour

#	Action	Rows it lifts	Effort
1	Write or recover <code>AGRITECH_PRD.md</code> and attach it as an appendix	1	1 h
2	Add the FR/NFR traceability matrix from Section 2 to the report	1	30 min

#	Action	Rows it lifts	Effort
3	Promote "Deliberate simplifications" into the report as <i>Scope Boundaries and Architectural Debt</i>	1, 2	20 min
4	Cut access-token TTL to 15 min and demo refresh	3	10 min
5	Add <code>@PreAuthorize</code> / an ADMIN interceptor in one service so authorisation is not header-string only	3	1 h
6	Add resilience4j circuit breakers plus gateway fallback URIs, then demo by killing a service	2, 4	3 h
7	Add springdoc OpenAPI aggregated at the gateway	4	30 min
8	Write <code>JwtAuthFilter</code> tests: accept, reject forged, reject revoked, strip spoofed headers	1, 3	1.5 h
9	<code>docker-compose.yml</code> for all seven services and show Eureka with two sensor instances	2, 4	3 h
10	Move H2 → PostgreSQL with one schema per service plus Flyway migrations	2	4 h
11	Add Micrometer Tracing + Zipkin and capture one four-hop trace screenshot	2, 4	2 h
12	Redis-backed rate limiting and revocation	3, 4	3 h

Items 1–5 are one working day and they are the difference between four 7s and four 9–10s. Items 6–12 are what you do if the review is more than a week away — and note that a reviewer cannot distinguish "not built" from "built but not mentioned", so **every item you do not build should appear in the report as a named, reasoned non-goal** rather than as a silence.

Demo script for the review room

Rehearse this in this order. It touches all four rows in about eight minutes.

- 1 Start the stack; show `stack.ps1` health-gating each service. (*Rubric 4 — operational config*)
- 2 Open Eureka at `:8761`; point at the registry showing all five instances. (*Rubric 2*)
- 3 Hit `/actuator/gateway/routes` on 8080 and read the live route table with `lb://` targets. (*Rubrics 2, 4*)
- 4 Log in as farmer, decode the JWT on `jwt.io`, point at `jti`, `role`, `farmId`, `exp`. (*Rubric 3*)
- 5 As farmer, attempt `POST /api/sensors/register` → 403. Same call as admin → 201. (*Rubrics 1, 3*)
- 6 Send a `curl` with a hand-written `X-User-Role: ADMIN` header and no token → 401, then with a valid farmer token → the header is stripped and the call still 403s. **This is the best ninety seconds of the whole demo.** (*Rubrics 3, 4*)
- 7 Ingest an out-of-range reading; show per-item validation rejecting it inside a valid batch. (*Rubric 1*)
- 8 Show a dry field go critical, open a valve, watch moisture and the health score recover, then watch the zone cut off at the optimal ceiling. (*Rubric 1 — the business requirement, not the plumbing*)
- 9 Logout, immediately replay the old token → 401 `token_revoked` within 15 s. Then state the 15-second window unprompted and name Redis as the fix. (*Rubric 3 — honesty about a limit scores higher than hiding it*)
- 10 Run `bash scripts/smoke-test.sh` and let the assertions scroll. (*all rows*)
- 11 Close on the Scope Boundaries table: what you cut, the ceiling of each cut, and the upgrade path. (*Rubric 1 — this is what "deep analysis" sounds like*)

Viva questions you should expect, with the answers

Why five services and not one? Because the scaling axes differ — sensor-service scales with device write volume, crop-service scales with CPU for analysis, irrigation-service scales with neither but is safety-critical and must be isolated. And because the agronomic model changes far more often than the device contract.

Why Eureka and not Consul or Kubernetes Services? Eureka is client-side load balancing with a locally cached registry, so it survives the registry dying — an AP trade-off that suits discovery. It is also the reference implementation for the Spring Cloud stack in use.

Where is the signing key held? Only the gateway and auth-service have it, and that is deliberate: auth mints, gateway verifies, and the four business services never parse a token at all. The weakness is that HS256 means both can mint; moving to RS256 lets the gateway hold only a public key.

How does a downstream service know the role is genuine? The gateway strips client `X-User-*` headers before writing verified ones, and services are meant to be reachable only through the gateway. The honest gap is that nothing currently *enforces* that network boundary — that is the answer to "so could you bypass it?", and it is better given proactively than found.

What happens if sensor-service is down when crop-service needs data? Right now: a 2–4 second timeout and a caught exception, degrading to whatever the response contract allows. The planned fix is a

resilience4j circuit breaker with a cached last-known-good snapshot. Naming the current behaviour accurately, then the fix, is a full-marks answer; claiming it is already resilient is not.

Why does the first request after startup fail? Eureka clients fetch the registry every 30 seconds, so for roughly 30 seconds the gateway has no route targets. That is the visible cost of AP eventual consistency in the registry, and it is why `stack.ps1` health-gates startup.

Why does the gateway poll auth-service instead of sharing its database? Because sharing a database across services would break the data-ownership boundary that makes the decomposition valid in the first place. A read-only polled endpoint keeps the boundary at the cost of a 15-second staleness window; Redis removes the window without removing the boundary.

What is `skipIfMoist` and why does it need two services? It is the water-saving rule: do not irrigate a field already at the optimal ceiling. The *knowledge* of optimal ceilings and current health belongs to crop-service; the *authority* to open a valve belongs to irrigation-service. Separating knowledge from actuation is precisely why the boundary exists.

Appendix — file inventory for the report

Cite these paths directly in your write-up; verifiable references are what make an analysis read as deep.

Claim	File
Single JWT verification point, header stripping, public paths	<code>api-gateway/src/main/java/com/agritech/gateway/JwtAuthFilter.java</code>
Route table, rate-limit tiers, CORS, actuator exposure	<code>api-gateway/src/main/resources/application.yml</code>
Token claims, HS256, 24 h TTL	<code>auth-service/src/main/java/com/agritech/auth/JwtService.java</code>
Credential check, role seeding, Google provisioning	<code>auth-service/.../AuthController.java</code> , <code>SeedData.java</code>
Supabase token introspection	<code>auth-service/src/main/java/com/agritech/auth/SupabaseVerifier.java</code>
Introspection tests against an embedded server	<code>auth-service/src/test/java/com/agritech/auth/SupabaseVerifierTest.java</code>
Role enforcement, fail-closed default	<code>common/src/main/java/com/agritech/common/CallerContext.java</code>
Uniform error contract	<code>common/.../ApiExceptionHandler.java</code>
Device registry, health thresholds, pagination	<code>sensor-service/.../SensorController.java</code> , <code>SensorDevice.java</code>
Telemetry validation and range queries	<code>sensor-service/.../TelemetryController.java</code>

Claim	File
Simulation and closed-loop watering	sensor-service/src/main/java/com/agritech/sensor/DeviceSimulator.java
Health scoring weights and band maths	crop-service/.../HealthScore.java + HealthScoreTest.java
Cross-service Eureka calls	crop-service/.../SensorClient.java, irrigation-service/.../FarmClient.java
Valve bounds, cut-out, emergency stop	irrigation-service/.../ValveController.java, IrrigationSchedulerTest.java
Startup order, health gating, env loading	scripts/stack.ps1
End-to-end scenario coverage	scripts/smoke-test.sh
Architecture, API tables, scope boundaries	README.md